

PORTFOLIO · PRODUCTION AI ENGINEERING

# An AI agent you can defend in an audit.

**Trust-Native ReAct Agent Framework** — a full-stack agent platform where layer boundaries are enforced by CI, every authorization flows through a signed trust kernel, and every decision is replayable.

**108K+**

lines of code,  
Python + TypeScript

**2,500**

test functions in  
186 test files

**8 wks**

first commit to  
deployable, solo

# Most agent demos die in security review

Typical agent frameworks ship a clever loop and leave trust, auditability, and security as an exercise for the reader. This codebase inverts that: safety is a property of the architecture, not a TODO.

1

## Architecture with teeth

A four-layer design where dependencies flow downward only. 12 dedicated architecture test modules mechanically verify the boundaries — an illegal import fails the build.

2

## A signed trust kernel

Authorization decisions flow through a pure, dependency-free kernel (stdlib + Pydantic only). Capability and policy records are cryptographically signed; changes force re-signing.

3

## A replayable audit trail

Black-box recordings, phase logs, and a signed agent-facts registry capture every LLM call, routing decision, and guardrail verdict — so “why did the agent do that?” has an answer.

# Four layers, enforced by the test suite

## ORCHESTRATION

LangGraph topology & state — thin wrappers only



## COMPONENTS

Domain logic: router, evaluator, plan builder — no LangGraph imports allowed



## SERVICES

Prompts, guardrails, LLM config, governance, memory, tools



## TRUST KERNEL

Pure types, protocols, crypto — zero framework deps, no I/O

*dependencies flow downward only*

## NOT A CONVENTION. A FAILING BUILD.

The **tests/architecture/ suite** contains 12 boundary-gate modules: dependency rules, service isolation, layer placement, read-only surfaces.

A component that imports the wrong layer breaks CI — the architecture cannot silently rot as the codebase grows.

# A 2:1 test-to-code ratio, by design

2,500

test functions

across 186 files, organized by architectural layer

53.6K

lines of test code

vs 26.7K lines of Python application code

0

live LLM calls in CI

model calls mocked by marker scope; live evals run out-of-band

12

architecture gates

boundary tests run as their own CI job on Python 3.13

## HOW IT'S BUILT

Property-based testing (Hypothesis) on the trust kernel · rejection tests written before acceptance tests for every guard · hash-pinned **requirements.lock** with a CI job proving a cold pinned install stays green · every prompt externalized as a Jinja2 template (23 templates, zero prompt strings in Python)

# Defense in depth, then prove it happened

1

## Input guardrail

LLM-as-judge rejects prompt injection & system-prompt overrides

2

## Tool validators

Deterministic Pydantic gates: shell command allowlist, path sandboxing

3

## Output guardrail

Scans for PII, API keys, and system-prompt leakage (regex + LLM)

4

## Injection classifier

Fourth line of defense in the governance layer

## EVERY DECISION, REPLAYABLE

### Black-box recordings

full input/output capture per run

### Phase logs

structured trace of each reasoning phase

### Signed agent-facts registry

capabilities & policies, GCS-backed in prod

### Explainability dashboard

6 read-only modules: traces, decisions, guardrails, agents, compliance, live logs

# Not a notebook — a deployable product

## FRONTEND

### 27.7K lines of TypeScript

- Next.js 15 · React 19 · CopilotKit UI
- WorkOS auth · Zod · Tailwind v4 / shadcn
- 75 frontend test files + read-only explainability app

## RUNTIME & MIDDLEWARE

### 26.7K lines of Python

- LangGraph orchestration · LiteLLM multi-provider routing
- Ports-and-adapters middleware: WorkOS JWT, Mem0 memory, Langfuse telemetry
- Postgres checkpointing · OpenAPI contract

## INFRA & DELIVERY

### 20 Terraform files, 10 Rego policies

- Cloud Run + Cloudflare edge + Neon Postgres dev tier
- Docker images for agent, backend, search
- 4 GitHub Actions workflows incl. wire-format baselines

WHAT THIS CODEBASE DEMONSTRATES

# Architecture leadership you can diff.

## Systems thinking

Four-layer design with mechanical enforcement — the kind of governance regulated AI teams need on day one.

## Test-first discipline

2:1 test-to-code ratio, property-based testing, failure-path-first guards.

## Regulated-AI fluency

Signed audit trails, explainability UI, OPA policies — built for the “explain this decision” conversation.

## Velocity, solo

146 commits, Apr 17 → Jun 10, 2026: runtime, frontend, middleware, and infra by a single engineer.

**Read the fine print (on purpose):** single-maintainer project, license pending, pre-1.0. Live-LLM evals run outside CI by design. Every figure here comes from a static audit of the repository — nothing is projected or extrapolated.